Ex: 06

Date: 6/8/26

# Error Detection & Correction - Hamming Code

**AIM:-**

To write a program to implement error detection and correction using Hamming code.

**Algorithm:**

1) Start the Program
2) Read the no.s of 4 bit data blocks (n) forming the input data stream.
3) For each 4 bit data block D = $d_1$ & $d_2, d_3, d_4$, Perform Hamming encoding.
   a) Arrange 7 bits with parity bit at position 1, 2 & 4 and data bits at position 3, 5, 6 and 7.
   b) Compute the parity bits using even parity:-
   $P_1 = d_1$ XOR $d_2$ XOR $d_4$ (cover bit - 1, 3, 5, 7).
   $P_2 = d_1$ XOR $d_3$ XOR $d_4$ (Cover bit - 2, 3, 6, 7).
   $P_4 = d_2$ XOR $d_3$ XOR $d_4$ (Cover bit - 4, 5, 6, 7).
   c) Form the 7 bit hamming code word: $P_1$ $P_2$ $d_1$ $P_4$ $d_2$, $d_3$ $d_4$.
4) Simulate transmission by introducing a single bit error at a chosen bit position (entered by user) in the code word
5) At receiver, compute the check bit (Syndrome) from received code word.

$C_1 = P_1$ XOR $d_1$ XOR $d_2$ XOR $d_4$

$C_2 = P_2$ XOR $d_1$ XOR $d_3$ XOR $d_4$

$C_4 = P_4$ XOR $d_2$ XOR $d_2$ XOR $d_3$ XOR $d_4$

6) Compute Error Position = $C_4 C_2 C_1$ (binary no formed)
7) Extract the original 4 data bit from corrected 7 bit code.

8.) Repeat steps 3 to 8 for all n blocks in the data.
9.) Stop the Program.

Program :-

```
def encode (d):
    """ d : list of 4 bits [d1, d2, d3, d4] -> returns
    7 bit hamming code list position (1-indexed) 1=P1,
2 = P2, 3 = d1, 4 = P4, 5 = d2, 6 = d3, 7 = d4 """.
        h = [0] * 7
h[2] = d[0]  # Position 3
h[4] = d[1]  # Position 5
h[5] = d[2]  # Position 6
h[6] = d[3]  # Position 7

h[0] = h[2] ^ h[4] ^ h[6]  # P1 Covers 1, 3, 5, 7.
h[1] = h[2] ^ h[5] ^ h[6]  # P1 Covers 2, 3, 8, 6, 7
h[3] = h[4] ^ h[5] ^ h[6]  # P4 Covers 4, 5, 6, 7
 return h
def detect and correct (h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]

error_Pos = c1*1 + c2*2 + c4*4  # Syndrone -> faculty bit
                                    Position.

if error_Pos != 0 :
    Print ("Error detected at bit position : error_pos)
    h[error_Pos - 1] ^ = 1          # flip error bit
else :-
    Print ("No error detected")
return error_Pos
```

=== HAMMING CODE - ERROR DETECTION AND CORRECTION

Enter number of 4-bit data blocks in the stream : 2

--- Block 1 ---

Enter 4 data bits (space separated, eg :- 1 0 1 1) : 0 1 0 1

Original Data bits : 0 1 0 1

Encoded Hamming code (7 bits) : 0 1 0 0 1 0 1

Enter bit position to introduce error (1-7, 0 = no error)

: 3

Transmitted code (with error induced at Pos 3) : 0 1 1 0 1 0 1

Received code : 0 1 1 0 1 0 1

Recovered data bits : 0 1 0 1

```
def to_str(bits):
    return "".join(str(b) for b in bits)

def main():
    n = int(input())
    for b in range(1, n+1):
        print("Block {b}")
        d = list(map(int, input().split()))
        h = encode(d)
    print("Original data bits : {to_str(d)}")
    print("Encoded hamming code (7 bits) : {to_str(h)}")
    pos = int(input("Enter bit to introduce error (1-7), 0 = "No error"))

    if 1 <= Pos <= 7:
        h[Pos-1] ^= 1
        print("Transmitted code : {to_str(h)}")
    else:
        print("Transmitted code : {to_str(h)}")
    print("Received code", {to_str(h)})
    print("Checking error")
    detect error and correct(h)
    print("Corrected code : {to_str(h)}")
    Td = [h[2], h[4], h[5], h[6]]
    Print("recovered data bits : {to_str(rd)}")
    if __name__ == "__main__"
        main()
```

Result :-

The program to implement error detection & correction using hamming code concept was successfully written in python, executed and verified with a sample input data stream.